

THE UNIVERSITY OF THE WEST INDIES



B.Sc. (Engineering)
Department of Electrical and Computer
Engineering

ECNG 3020 – SPECIAL PROJECT
**Self-Contained Safety App using
Spatio-Temporal Anomaly Detection**
FINAL REPORT

Denali Mc Knight
816032736

April 1, 2025

Project Supervisor: Dr. Patrick Hosein

Project Type: II

ELECTRICAL
& COMPUTER
ENGINEERING
DEPARTMENT



THE UNIVERSITY OF THE WEST INDIES
ST. AUGUSTINE, TRINIDAD & TOBAGO, WEST INDIES
FACULTY OF ENGINEERING
Department of Electrical & Computer Engineering
B. Sc. in Electrical & Computer Engineering

CHEATING, PLAGIARISM AND COLLUSION DECLARATION FORM

According to Rules 3.31 and 3.32 of The UWI Faculty of Engineering Undergraduate Regulations and Syllabuses 2018/2019:

3.31 "Cheating, Plagiarism and Collusion are serious offences under University Regulations.

- (a) Cheating is any attempt to benefit one's self or another by deceit or fraud.
- (b) Plagiarism is the unauthorised and/or unacknowledged use of another person's intellectual efforts and creations howsoever recorded, including whether formally published or in manuscript or in typescript or other printed or electronically presented form and includes taking passages, ideas or structures from another work or author without proper and unequivocal attribution of such source(s), using the conventions for attributions or citing used in this University. Plagiarism is a form of cheating.
- (c) For the purposes of these Regulations, 'collusion' shall mean the unauthorised or unlawful collaboration or agreement between two or more students in the preparation, writing or production of a course assignment for examination and assessment, to the extent that they have produced the same or substantially the same paper, project report, as the case may be, as if it were their separate and individual efforts, in circumstances where they knew or had reason to know that the assignment or a part thereof was not intended to be a group project, but was rather to be the product of each student's individual efforts.

3.32 Cheating, plagiarism and collusion shall be reported to the Campus Committee on Examinations and the penalties would be in accordance with the University Examination Regulations."

I,Denali Mc Knight....., have read and understood Rules 3.31 and 3.32 of The UWI Faculty of Engineering Undergraduate Regulations and Syllabuses 2018/2019 on Cheating, Plagiarism and Collusion.

I understand that my submission is subject to the electronic plagiarism checker, Turnitin.

I declare that this assignment is my own work and does not involve cheating, plagiarism or collusion.

Signature:.....

Date:.....1/4/2025.....

Abstract

The lack of safety is a growing problem as there has been an increase in missing persons due to kidnappings and other criminal activity. While there are applications that assist with this problem, they require location data to be transmitted over the internet. The project proposes a self-contained spatiotemporal anomaly detection algorithm that was implemented and tested using an Android application. The algorithm designed, uses a semi supervised, temporal cluster-based, distance-based approach. The algorithm detects spatiotemporal anomalies and utilizes user feedback to learn the user's movement. The mobile application developed was tested in real-world applications using beta testers as they did their day-to-day activities. Results show that the algorithm successfully detected anomalies however, the mobile application was restricted by the Android Operating System after running in the background for longer than a day.

Table of Contents

List of Figures	vi
List of Tables	vii
List of Abbreviations	viii
1. Introduction	1
1.1. Project Objectives and Scope.....	2
2. Background Theory and Related Work.....	4
2.1. Cluster Methods	6
2.2. Classification Methods.....	6
2.3. Deep Learning.....	7
2.4. Others.....	7
2.5. Supervised vs Unsupervised Algorithms	8
3. Methodology	9
3.1. Requirements	9
3.2. Software Development Life Cycle (SDLC) Model	12
3.3. Programming Language Selection.....	12
3.4. IDE Selection.....	13
3.5. Overview of Algorithm.....	15
3.5.1. Spatiotemporal Data Storage	16
3.5.2. Pre-processing phase.....	16
3.5.3. Temporal Clustering	17
3.5.4. Spatial Aspect	18
3.5.5. Threshold Calculation.....	19
3.5.6. Temporal Aspect.....	20
3.5.7. Temporal Error Refinement.....	21
3.5.8. Temporal Boundary Considerations	22
3.5.9. Temporal Algorithm	22
3.6. Overview of Mobile Application.....	24
3.6.1. Location Client Selection.....	25
3.6.2. Location Data Acquisition	26
3.6.3. User Alerting.....	27
3.6.4. SOS Message	27
3.6.5. Alert Snoozing	27
4. Results	28
4.1. Testing Methodology	28
4.1.1. Unit Testing	28
4.1.2. Integration Testing	29
4.1.3. Alpha Testing.....	29
4.1.4. Beta Testing	29

4.2.	Traceability Matrix	30
4.3.	Location Acquisition Module	31
4.4.	Route Storage Module	32
4.5.	Spatial Anomaly Detection Module.....	34
4.6.	Temporal Anomaly Detection Module	36
4.7.	Notification Module.....	38
4.8.	SOS Message Module.....	40
4.9.	Alert Snoozing Module.....	44
4.10.	Alpha Testing.....	46
4.11.	Beta Testing	48
4.12.	Battery Consumption	51
4.13.	Storage Usage	54
5.	Discussion	55
5.1.	Spatiotemporal Anomaly Detection Algorithm Sufficiency.....	55
5.1.1.	Algorithm Improvements.....	56
5.2.	Battery Optimization Issues	57
5.2.1.	Android's Adaptive Battery	57
5.2.2.	Capacity of phones.....	59
5.3.	Frequency of Location Updates	59
5.3.1.	Background Location Access	60
5.3.2.	Location batching.....	60
5.3.3.	Data connection	61
5.3.4.	Location Update Mode/Priority	62
5.4.	START_STICKY Flag	62
5.5.	Possible Solutions	63
5.6.	Alert Snoozing Module.....	63
5.7.	Application Improvements.....	64
6.	Conclusion.....	65
	References.....	66
	Appendix A: Routes for Spatial Testing.....	68

List of Figures

Figure 1: Diagram of Agile Model	12
Figure 2: Diagram of Algorithm	15
Figure 3: Diagram of Scenario with 90-degree bend in road.....	20
Figure 4: Flowchart of Application.....	25
Figure 5: Screenshot of Notification.....	40
Figure 6: Screenshot of SOS message with (0.0,0.0) Location	43
Figure 7: Screenshot of SOS message working as expected.	44
Figure 8:Screenshot of Application Dialog to Snooze Alerts.....	45
Figure 9: Screenshot of Application Percentage Battery Usage on Balanced Power mode	51
Figure 10: Screenshot showing Breakdown of Battery Usage	52
Figure 11: Screenshots showing Application Percentage Battery Usage in High Accuracy Mode	53
Figure 12: Screenshot of Application Storage Usage	54
Figure 13: Screenshot of Unrestricted Battery Setting	58
Figure 14: Screenshot showing Application added to "Never Auto Sleeping" List	59
Figure 15: Screenshot showing Background Location Permission	60

List of Tables

Table 1: Summary of Spatio-Temporal Outlier Detection Methods (Wu and Tian 2020).....	4
Table 2: Functional Requirements of the Algorithm	9
Table 3: Non-Functional Requirement of the Algorithm	9
Table 4: Functional Requirements of the Application	10
Table 5: Non-Functional Requirements of the Application.....	11
Table 6: Requirements Traceability Matrix	30

List of Abbreviations

API – Application Programming Interface
FLP – Fused Location Provider
IDE – Integrated Development Environment
OS – Operating System
PoC – Proof of Concept
PT – Point's Time
PTFS – Point's Time from Start
RAM – Random Access Memory
RLTFS – Route Line's Time from Start
SOS – Save Our Souls
TEV – Temporal Error Variable
TFS – Time from Start

1. Introduction

Safety has become a major concern worldwide. In Africa, the continent with the most missing person, there has been a 75% increase in the last 5 years (International Committee of the Red Cross 2024). In Haiti, in 2023, there was a 300% increase in the kidnapping of women and children in comparison with 2021 (UNICEF 2023). In Trinidad, there has been six reported kidnappings for ransom in 2024 against 2023's three. 26% of missing persons are still unaccounted for in 2024 and 17% in 2023. A decade ago in 2011, this number was 3%. The statistics therefore reflects this growing concern. (Daily Express 2024)

There are multiple applications developed that address this issue such as Life360, Glympse, Google maps, GeoZilla, etc. These applications utilise a real time location sharing method where a person's real-time location is broadcasted to their group members and they can monitor the location data and determine if they may be in a foreign or unusual location. Applications also allow alerts to be sent to all group members if a person leaves a preset geofenced location. Users can also send SOS alerts to their group members if they suspect that they are in an unsafe situation. They also allow users to access their group members' location history and provide some basic analysis of the history such as favourite route. Some even provide more advanced analysis like unsafe driving habits.

However, these applications provide a manual or semi manual SOS alerts triggering with automatic alerts only being sent when the user leaves a geofenced location. A person's live location is also shared over the internet which raises privacy concerns for potential users. In 2025, there has been a reported incident where a US company that tracks and sells Americans' location data (Collier 2025). The company, Gravy Analytics, has since been accused of selling location data illegally by the US Federal Trade Commission and has been banned from selling the data (Federal Trade Commission 2025). There is also an increased risk of a user's location

data being breached as the location history is shared with the group, meaning that there are more points of vulnerability. These applications also require internet access which may be inaccessible while they are travelling or in dead zones.

Considering this, the project proposes a mobile application that utilizes a Self-Contained Spatio-Temporal Anomaly Detection Algorithm. This application will learn a user's location pattern and detect when the user deviates from the learned locations. It will deviate from existing applications as it will not require the user's friends and family to actively monitor the user's live location. Instead, the app will detect when the user deviates either spatially or temporally and allow the user to state if they are safe. If the user does not state that they are safe after a specified period, then an SOS alert is sent to the user's emergency contacts. The application will be self-contained and therefore does not necessitate the internet access and greatly reduces the risk of a location data breach. The algorithm will also be sufficiently lightweight and efficient to minimise resource utilization while constantly running in the background.

1.1.Project Objectives and Scope

The objectives of this project are as follows:

- Develop a spatio-temporal anomaly detection algorithm.
- Develop a Proof-of-Concept mobile application using this algorithm.
- Perform beta testing with friendly users.

This project is only a proof of concept and will focus on the spatio-temporal anomaly detection algorithm. The proof-of-concept app will be developed for Android OS and will be tested on a relevant mobile device. It will access the location capabilities of the device and will therefore be tested in Trinidad and Tobago.

This report is organized as follows: 1. Introduction that highlights the issue the application seeks to address and provides the aim and scope of the project. 2. Background Theory and

Related Work compares the methods and approaches for spatiotemporal anomaly detection. 3. Methodology itemizes the requirements of the project and justifies the choices made throughout the design and development of the project. 4. Results highlights the methods used in testing that the results that were obtained. 5. Discussion provided analysed the results obtained and offered probable reasons for the results. 6. Conclusion summarises the results obtained in testing.

2. Background Theory and Related Work

A general definition of anomaly detection is the identification of behaviours that do not conform to expected and accepted behaviour (Nawaratne, et al. 2019). A spatial-temporal anomaly is an object whose behavioural (non-spatial and non-temporal) attributes are significantly different from those of the other objects in its spatial and temporal neighbourhoods (Karadayi, Aydin and Ogrenci 2020). In the context of this project, an anomaly will be when the user deviates from their usual learned movement patterns.

The many existing spatiotemporal anomaly detection methods can be categorized in the following: (Wu and Tian 2020)

1. Cluster-based approaches use existing and popular cluster algorithms to group the set of data into several data clusters based on similarities to find an outlier.
2. Classification-based approaches divide the data set to form Gaussian (Normal) distribution data sets and detect outliers via existing classic classification algorithms.
3. Deep learning approaches use various deep learning or machine learning algorithms to find the outliers of data set.
4. Other approaches may use unique method which are likely to include the categories above.

Table 1: Summary of Spatio-Temporal Outlier Detection Methods (Wu and Tian 2020)

Category	Method
Cluster	Partition-based
	Hierarchy-based
	Density-based/Distance-based
	Grid-based
	Model-based

	Fuzzy-based Neighbour-based
Classification	Support Vector Machines (SVM) Bayesian Belief Network ST Classification
Deep Learning	Regression Kernel Genetic Algorithm Dimension Reduction Hadoop Wavelet Transform Rough set Fusion-based Convex Polytope
Others	ST Correlation Smooth Spatial Attribute Method Boxplot-based Define-based Computer-based

2.1.Cluster Methods

One study used a partition-based method to detect outliers in taxi trajectories. Trajectories with the same starting and ending points were extracted from the data and partitioned into segments. The similarity between the trajectories based on distance and density were calculated and anomalies were detected if the similarity did not fall within a certain threshold. The method successfully extracted personalized routes as well as abnormal parking location and traffic areas from the anomalous trajectories (Yan, et al. 2017). A similar study was also conducted, partitioning based on minimum description length (MDL) principle and clustering using a density-based clustering algorithm. This clustering method reduced the computational time of the outlier detection (Eldawy and Mokhtar 2020).

A popular distance-based method is the Local Outlier Factor (LOF) method. This method assigns a numerical value to each data point based on its distance to its neighbours and its density. This value is the Local Outlier Factor. A high factor usually denotes that the point is an outlier (Salehi, et al. 2016). Another study combined this method with the Density-Based Spatial Clustering of Applications with Noise (DBSCAN) method to detect spatiotemporal outliers in bathymetry data. This method involved utilizing LOF to find the local outliers for each spatial region and then DBSCAN to find the temporal clusters of the outliers (Sedaghat, Hersey and McGuire 2013).

2.2.Classification Methods

The Support-Vector Machine (SVM) method has received great interest from the machine learning community since their introduction. One study designed a maritime anomaly detection algorithm based on an SVM. It used a novel feature-based extraction method which mapped trajectories onto a high dimensional space which accounted for both spatiotemporal and motion features. It then utilized a density-based spatial clustering to recognize trends in the data.

Afterwards, an improved SVM with a weighted hybrid kernel function was used to detect anomalies. It was superior to a standard SVM in that it can revise the kernel function to obtain the most optimal based on the set (Wei, Xie and Zhang 2022).

2.3. Deep Learning

A study utilized a Hadoop-based spatiotemporal outlier detection algorithm. This was proposed as the existing methods at the time could not deal with big data so a big data processing platform, Hadoop, was used. The algorithm involved detecting the spatial outliers using a weight-based algorithm and then identifying if the detected spatial outliers were also temporal outliers. The study concluded that the Hadoop based algorithm functioned better than the approach without Hadoop (Yao and Wang 2015).

2.4. Others

One study proposed a spatiotemporal anomaly detection network for multivariate time series using a predictability method (Tian, et al. 2023). Tian states that the unavailability of labelled datasets, the complexity of the sensor relationships and inconspicuousness of anomalies in high dimensional spaces poses challenges for spatiotemporal anomaly detection in multivariate time series. To address these challenges, the study proposes a solution with two main modules, the predictive module and the error refinement module. The predictive module learns the relationships between sensors, encodes this data as a graph. This data is then aggregated with the historical data of the sensor to predict the future behaviour of each sensor. The error refinement module reconstructs the prediction error to refine the prediction value. This improves the model's ability to distinguish between anomalies and regularities.

2.5.Supervised vs Unsupervised Algorithms

Anomaly detection algorithms can be supervised or unsupervised. Supervised techniques utilize labelled data (the training set) to classify each anomaly. Its drawbacks include the necessity for labelled data which can be tedious in many real-world applications and the inability to detect new and rare anomalies. On the other hand, unsupervised methods detect outliers as data points which vary greatly from the majority of the data (normal data) (Pokrajac, Lazarevic and Latecki 2007). A drawback of these traditional methods is that they focus on the density of an object to its neighbours while ignoring the similarity which means that normal but sparse data can be incorrectly labelled as outliers (Zhang, et al. 2025). Semi-supervised approaches combines both supervised and unsupervised by using both labelled and unlabelled data to train the algorithms (Bergmann 2023). Arses et al., describes a semi-supervised anomaly detection approach as an unsupervised anomaly detection model which is trained with only healthy (normal) data. They implemented a novel semi-supervised spatiotemporal anomaly detection algorithm for the physics particle reading channels of a hadronic calorimeter. It utilized convolutional and graph neural networks to learn the spatiotemporal data. The algorithm successfully detected anomalies with a production level performance and was implemented into the current calorimeter system (Asres, et al. 2023).

3. Methodology

3.1. Requirements

To sufficiently address the problem highlighted, the algorithm designed should satisfy the following requirements.

Table 2: Functional Requirements of the Algorithm

Functional Requirements	
Requirement ID	Description
FR-AL-1	The algorithm should accept and store incoming location data with its timestamps.
FR-AL-2	The algorithm should detect movement from incoming location data.
FR-AL-3	The algorithm should determine and itemize routes.
FR-AL-4	The algorithm should learn movement and improve on stored knowledge.
FR-AL-5	The algorithm should identify spatial anomalies.
FR-AL-6	The algorithm should identify temporal anomalies.
FR-AL-7	The algorithm should accept user input to determine whether to store or discard location data.

Table 3: Non-Functional Requirement of the Algorithm

Non-Functional Requirements	
Requirement ID	Description

NR-AL-1	The algorithm should be computationally light weight to complete execution within 50% of the location update interval.
NR-AL-2	The algorithm should not utilize more than 10% of device's battery in 24 hours.
NR-AL-3	The algorithm should detect spatial anomalies with 95% accuracy
NR-AL-4	The algorithm should detect temporal anomalies with 95% accuracy
NR-AL-5	The algorithm should not utilize more than 100MB of storage after a month of use.
NR-AL-6	The algorithm should account for acceptable inaccuracies and errors in spatial data.
NR-AL-7	The algorithm should account for acceptable temporal variations in movement pattern data.
NR-AL-8	The algorithm should account variations in traffic density

The algorithm will be implemented in a mobile application to be tested in simulation and in real-world scenarios. The application developed also has requirement that should be satisfied to ensure that the algorithm is comprehensively tested and the user experience is satisfactory.

Table 4: Functional Requirements of the Application

Functional Requirements	
Requirement ID	Description
FR-AP-1	The application should provide location data with timestamps at regular intervals.

FR-AP-2	The application should provide the user interface for the user to start and stop the algorithm from running.
FR-AP-3	The application should provide interface to notify users when anomalous data is detected.
FR-AP-4	The application should allow the user to indicate whether movement is anomalous or not.
FR-AP-5	The application should allow the user to add and delete Emergency contacts
FR-AP-6	The application should automatically send SOS messages when anomalous data is detected and the user does not indicate that data is not anomalous.

Table 5: Non-Functional Requirements of the Application

Non-Functional Requirements	
Requirement ID	Description
NR-AP-1	The application should provide location data with an accuracy of at least 100m
NR-AP-2	The application should account for intermittent inaccuracies of the algorithm.
NR-AP-3	The application should not utilize more than 10% of device's battery in 24 hours.
NR-AP-4	The application should not utilize more than 100MB of storage after a month of use
NR-AP-5	The application should be able to run in the background

NR-AP-6	The application should not utilize more than 100 MB of RAM at any given time while running in the background.
NR-AP-7	The application should allow users to snooze the alerts for a set period

3.2. Software Development Life Cycle (SDLC) Model

The Software Development Life Cycle Model used was the Agile model. Given the size of the project and that there was one developer, this model was most suitable. This model allowed for gradual and incremental development which allowed bugs to be found and rectified earlier in development. This also enabled the near seamless integration of upgrades and additional features as each round of development was followed by a round of testing.



Figure 1: Diagram of Agile Model

3.3. Programming Language Selection

Two main programming languages are used for Android development: Kotlin and Java. Java has existed as the original language for Android programming and therefore has a large base of libraries and community support. It also provides cross platform compatibility and is widely used beyond Android development.

However, Kotlin was developed by JetBrains in 2010 to solve some of the issues that was experienced with Java such as lack of support for non-nullable types (JetBrains 2025). Kotlin also sought to reducing the length of code and make coding more efficient. Kotlin provides a large host of modern features and is completely interoperable with Java. In 2019, Kotlin was declared by Google as the official language for Android development (Google Inc. 2024). Google has also provided many tutorials and courses for new Android developers. Kotlin was therefore selected as the programming language for development in this project.

3.4.IDE Selection

Four main IDEs were suggested during research for the IDE to be used for development. They are:

1. Android Studio
2. IntelliJ IDEA
3. Visual Studio + Xamarin
4. Eclipse

These were evaluated by the following criteria to select the most suitable for this project.

Criteria	Android Studio	IntelliJ IDEA	Visual Studio + Xamarin	Eclipse
Smart Code editor	Yes	Yes	Yes	Limited
Kotlin Support	Yes	Yes	Yes	Yes
Integrated Android emulator	Yes	No	No	No
Debugger	Yes	Yes	Yes	Yes

Version Control manager/GitHub integration	Yes	Yes	Yes	Yes
Android Debug Bridge Integration	Yes	No	No	No
Cost	Free	Free community version	Free	Free
Google's Endorsement	Yes	No	No	No

Therefore, IDE used for development was the Android Studio IDE (Version: Android Studio Ladybug Feature Drop | 2024.2.2 Patch 2; Build #AI-242.23726.103.2422.13103373, built on February 21, 2025). This is the official IDE for Android development built by Google. This IDE provided an intelligent code editor that allows for code completion, real-time error detection and syntax highlighting. It also provided a debugger and Android Emulator which significantly assisted in testing. It also facilitated the creation of the APK which could be downloaded by any Android device. Its GitHub integration was used for the version control for the project. The code of the project was uploaded to the following link: https://github.com/denali2905/SC_STAD.

3.5. Overview of Algorithm

This project proposes a Self-contained, semi-supervised, distance-based spatiotemporal anomaly detection algorithm with basic machine learning capabilities and a basic error refinement module. The algorithm also relies on some user feedback. The algorithm is designed to detect spatial and temporal anomalies based on the learned movement pattern up to that point in time. The user is presented with the anomalies as they occur, where they can select whether the data is truly anomalous or if it is normal but just unknown at that point.

The user's learned movement patterns are processed and stores as a series of straight-line equations with timestamps. As the user moves, their location points is are tested to see if it lies on any of the lines stored. If the point lies on a line, the timestamp of the user's current location is compared with the line's timestamp to see if it falls within a certain threshold. If point satisfies both criteria, the point is treated as normal data, otherwise the algorithm treats this point as an anomaly. A temporal error variable is also implemented to account for the daily variations in traffic.

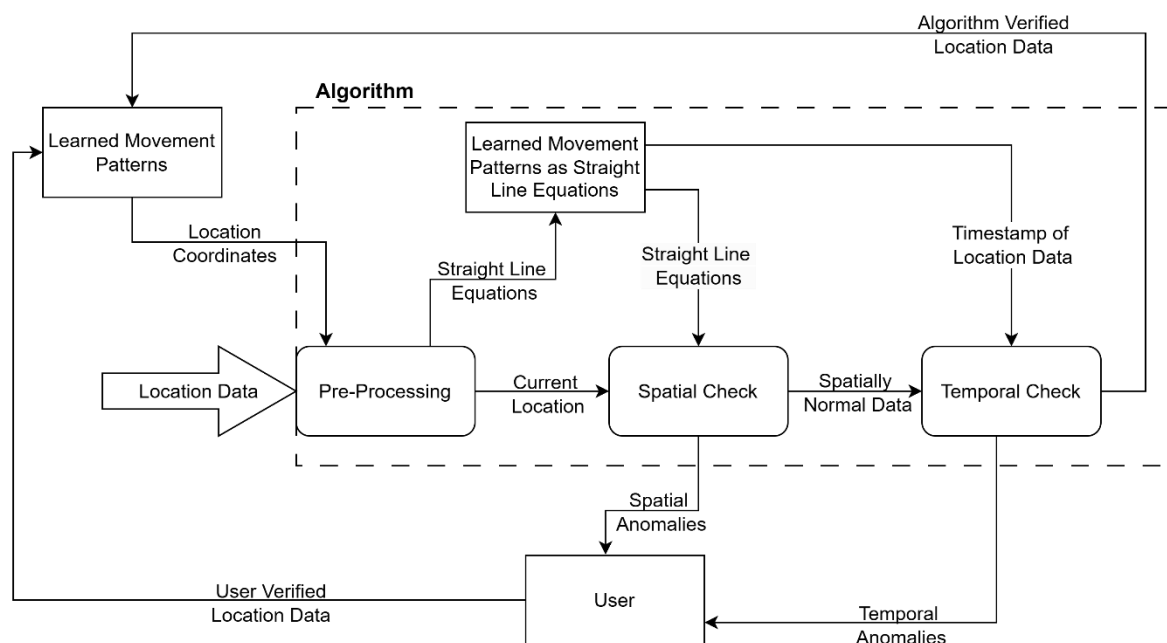


Figure 2: Diagram of Algorithm

3.5.1. Spatiotemporal Data Storage

The user's location data is stored in two files: A Master file and a Learned Routes file. The Master file contains the raw latitude and longitude points as decimals and the timestamps in UNIX Epoch time. The file contains all the data points, both anomalous and normal, serves as a backup and is used for export if required. The Learned Routes file contains the normal data stored as groups of the coordinate points and timestamps called routes. A route point as coined in this project is a data structure in this application that contains the latitude and longitude coordinate points and timestamps for any one data point. A route is collection of a continuous series of route points for movement in a period without any extended period of spatial rest, i.e., a group of data points for a period of movement.

As each data point is received, it is tested against the last received data point to see they are approximately the same. If they are, the location point is not stored again on the files but the timestamp of the last point is updated only in the Current Route data structure in the application. This is to reduce the storage space utilized and reduce computational expense as only relevant data is stored.

3.5.2. Pre-processing phase

At the initialisation of the application, the Learned Routes file is read into a series of routes. These routes are then converted to route lines which is a data structure that contains the two coordinate points and an average of the two timestamps and the time in milliseconds from the first point of the route, referred to as the time from start (TFS). The data structure contains member variables that calculates the y-intercept, c , and gradient, m , the day, hour and minute

of the time stamp and 24-hour time converted to minutes. The group of route lines is stored as long as the application is running so there is no need to calculate every time a new point is received. When a new route is being stored, its route lines are also calculated and stored alongside the others.

$$y = mx + c \quad [1]$$

$$m = \frac{lon_1 - lon_2}{lat_1 - lat_2} \quad [2]$$

$$c = lon_1 - m(lat_1) \quad [3]$$

$$time\ in\ minutes = hour * 60 + minute \quad [4]$$

$$time\ from\ first\ route\ point = time_{route\ point} - time_{first\ route\ point} \quad [5]$$

where lon_x and lat_x are the coordinates of route point_x,

m is the gradient,

c is the intercept,

$hour$ is the hour of the timestamp in 24hr format,

$minute$ is the minute of the timestamp,

$time_{route\ point}$ is the timestamp in UNIX epoch time in milliseconds

$time_{first\ route\ point}$ is the timestamp of the first point in UNIX epoch time in milliseconds

3.5.3. Temporal Clustering

The learned routes are clustered to reduce the computational time of the algorithm. The routes are clustered based on if it falls on a weekday or a weekend. A weekday includes the days from Monday to Friday and the weekend days are Saturday and Sunday. These groups were used as the most work weeks and school weeks span from Monday to Friday. These days are likely to follow the same routine for many users. Clustering by each day of the week was also considered but decided against due to the extended period of time the algorithm would take to learn a user's various movement patterns. Clustering reduced the amount of data that the algorithm must sift through thereby reducing computational load.

Spatial clustering by starting point was also considered and implemented but removed as users may make detours and start a route from a different starting point but then merge onto a normal route at a normal time. With this type of spatial clustering, this will be flagged as an anomaly and may be viewed as a malfunction of the algorithm to a user.

3.5.4. Spatial Aspect

When the algorithm detects that the user begins to move, the received data points is checked for anomalies given that the user has routes already learned. If there a no learned routes, no checks are done.

Based on the day, the data point is checked against all the routes in that cluster as explained in the subsection above. If the point is found on one of the route lines, it is passed on to the temporal aspect of the algorithm.

The spatial check is done by taking the gradient of a line and the coordinates of the point to determine a y-intercept using equation [3]. The intercept calculated is compared against the intercept of the line to determine if it lies close to or on the line within a threshold calculated in the next subsection. The point is then checked to see if it falls within the bounds or close to the bounds of the line's latitude and longitude points. This is to reduce the risk of false positives as a point may have the same gradient of a line but lie far from the actual line.

1. For each line in route:
 - a. Calculate y-intercept using equation [3, gradient of line and coordinate point
 - b. If y intercept lies within threshold
 - i. YES: Go to step c.
 - ii. NO: Go to next line
 - c. If latitude point lies in between or near route line's latitude points
 - i. YES: Go to step d.

- ii. NO: Go to next line
- d. If longitude point lies in between or near route line's longitude points
 - i. YES: Go to temporal aspect.
 - ii. NO: Go to next line
- 2. If point is not found, return false.

3.5.5. Threshold Calculation

Google states that the approximate location using their API is usually with 50 metres (Google Inc. 2025). This is roughly 0.00045 degrees of latitude and 0.00085 degrees of longitude. Using this as an error, sample points and the error propagation method, the threshold was estimated as follows:

$$m_{relative\ error} = \frac{lon_{1error} + lon_{2error}}{lon_1 - lon_2} + \frac{lat_{1error} + lat_{2error}}{lat_1 - lat_2} \quad [6]$$

$$m_{relative\ error} = \frac{0.00085 + 0.00085}{|61.5225454 - 61.5203279|} \quad [7]$$

$$+ \frac{0.00045 + 0.00045}{|10.6630934 - 10.6623897|}$$

$$m_{relative\ error} = 0.7666 + 1.2790 = 2.0456 \quad [8]$$

$$c_{relative\ error} = m_{relative\ error} + lat_{error} + \frac{lon_{error}}{lon} \quad [9]$$

$$c_{relative\ error} = 2.0456 + \frac{0.00085}{|61.5215765|} = 2.0456 \quad [10]$$

$$c_{error} = 2.0456 * c = 27.92149 \quad [11]$$

This error is large and impractical; therefore, the threshold was estimated using during testing and was set at ± 0.001 . The boundaries around the latitude and longitude points were set at

± 0.0015 which accounted for the absolute error of the reading and the fact that location points could be as far as 100 metres away from the line it lies on. As points are not taken every second, the 90-degree corners of roads will not be exactly accounted for. As in Figure 3, the user's position will be considered anomalous if the boundaries of the lines are kept exact. With the extended boundaries, the user position is considered normal as it should be. This threshold was also refined in testing.

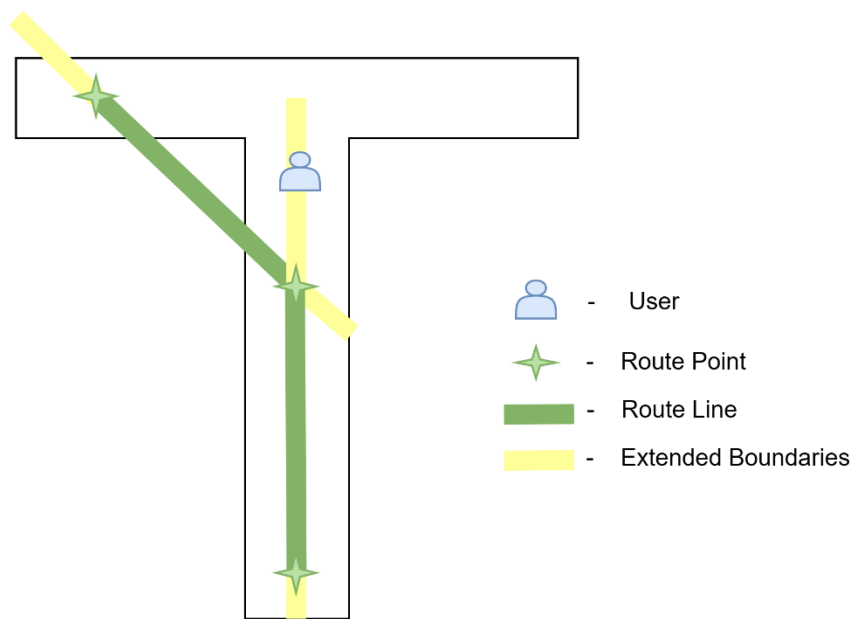


Figure 3: Diagram of Scenario with 90-degree bend in road

3.5.6. Temporal Aspect

When a data point is confirmed as spatially normal, its temporality is then tested for anomalies. The absolute time of the data point is compared to the timestamp of the route line that the data point was found to spatially lie on. If the time of the point is within ± 15 minutes of the timestamp of the route line, it is considered as normal and the algorithm returns true. Additionally, if the time does not satisfy this criterion, the time of the route point from the start of the route is also compared against the time from start (TFS) for the route line. The hour of the point and the route line is compared to see if they are within an hour of each other. The

TFS for route point and route line is then checked to see if they are within 15 minutes of each other and returns true if it does.

The purpose of conducting both types of checks is to account for most situations reducing the number of false negatives returned. The absolute time comparison accounts for most situations. One specific scenario it accounts for that the other method does not account for is if the user has a detour to make and leaves at an earlier time to then merge back onto their normal route at the normal time. The TFS method would not consider this as normal data as the TFS of the point would be inflated. The TFS method gives more range to temporal test as it accounts for scenarios where the user may start a normal route more than 15 minutes away from the normal time but not more than an hour away. This is a common occurrence as persons may run late or leave early for various reasons. This method does not reduce the integrity of the temporal check as it keeps the threshold as 15 minutes from the TFS of the route line.

3.5.7. Temporal Error Refinement

A Temporal Error Variable (TEV) is utilised to further reduce the chance of false negatives. The TEV accounts for the daily variations in traffic density. It is also common that the traffic varies on a day-to-day basis due to accidents, roadblocks, holidays, etc. The TEV is the difference between the normal route time and the user's current time. This is calculated and stored when the user's time is found as normal. This is then factored into the subsequent temporal checks for both methods. This works by shifting the threshold to account for the previous difference between the user's current time and the normal route's time. Each route has its own TEV.

3.5.8. Temporal Boundary Considerations

Given that time goes from 0:59 to 1:00 and 23:59 to 00:00, special considerations must be given for timestamps nearing these boundaries. To eliminate the hour-to-hour boundary issue, the timestamps were converted to minutes with equation [4]. With this, the timestamps became a continuous incremental series of natural numbers for the same day.

However, there was still the need for special consideration for different days. For instance, if a timestamp for a route line was at 23:55 on day 1, the threshold should lie from 23:40 on day 1 to 0:10 on day 2. This necessitates a piecewise check to see if the time lies in 23:40 to 23:59 or in 0:00 to 0:10. There was also need to consider whether the TEV would cause the timestamp to exceed these boundaries as well.

3.5.9. Temporal Algorithm

The algorithm of the temporal aspect with these considerations is below. The point's time (PT) refers to the timestamp of the point being tested and the route line's time (RLT) refers to the timestamp of the route line that the point lies on. These times are in the time in minutes format. The TFS of the point is referred to as PTFS (point's time from start) and the route line's TFS as RLTFs. These are in milliseconds. The % operator refers to the modulo operator.

1. If RLT is between 15 and 1424
 - a. YES: go to step 2
 - b. NO: go to step 3
2. If $(PT + TEV) \% 1439$ is in range from $RLT - 15$ to $RLT + 15$
 - a. YES: Calculate TEV as $RLT - PT$ and return true
 - b. NO: go to step 9
3. If RLT is less than 15
 - a. YES: go to step 4

- b. NO: go to step 6
- 4. If $(PT + TEV + 1440) \% 1439$ is in range from RLT-15 to 1439
 - a. YES: Calculate TEV as $RLT - PT + 1440$ and return true
 - b. NO: go to step 5
- 5. If $(PT + TEV + 1440) \% 1439$ is in range from 0 to RLT+15
 - a. YES: Calculate TEV as $RLT - PT$ and return true
 - b. NO: go to step 9
- 6. If RLT is greater than 1424
 - a. YES: go to step 7
 - b. NO: go to step 9
- 7. If $(PT + TEV + 1440) \% 1439$ is in range from RLT-15 to 1439
 - a. YES: Calculate TEV as $RLT - PT$ and return true
 - b. NO: go to step 5
- 8. If $(PT + TEV + 1440) \% 1439$ is in range from 0 to $(RLT + 15) \% 1439$
 - a. YES: Calculate TEV as $RLT - PT - 1440$ and return true
 - b. NO: go to step 9
- 9. If $PTFS + TEV * 60,000$ is in range from $RLTFS - 900,000$ to $RLTFS + 900,000$.
 - a. YES: Calculate TEV as $(RLTFS - PTFS) / 60,000$ and return true
 - b. NO: go to step 10
- 10. Return false

3.6.Overview of Mobile Application

A Proof-of-Concept (PoC) application was developed to test the algorithm. This was developed for android rather than IOS due to availability of devices. As, the application was only a PoC, the aesthetics were not focused on.

The application developed allowed the user movement to be tracked and stored in the background and provided notifications to alert the user when they deviated from the learned routes for an extended period. This notification allows user to interact and state whether they are safe. If left unattended for another extended period, the user's emergency contacts are alerted. The application also allows the user to add and delete their emergency contacts. The user can also snooze the alerts for a desired period and send an SOS message to their emergency contacts at any point with the click of a button in the app or from a notification.

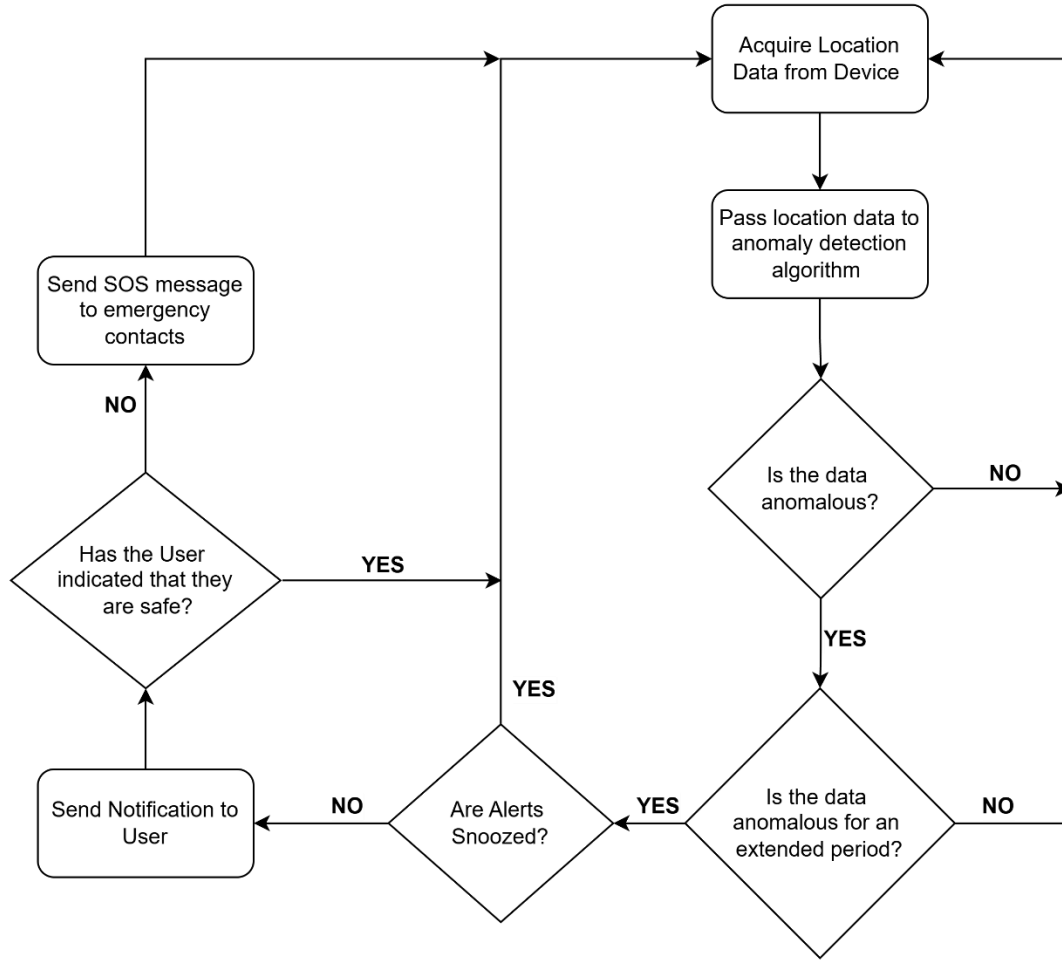


Figure 4: Flowchart of Application

3.6.1. Location Client Selection

The user's location data was acquired using Google's FusedLocationProviderClient. This API provided a much more power efficient method of retrieving location data in comparison to a device's GPS provider and a more accurate method than a Network location provider. This API "fuses" GPS with mobile network and Wi-Fi location technology to provide location data based on the need. It allows the developer to specify if they require high accuracy at the expense of battery power or low power at the expense of accuracy. It also provides a balanced power mode with equal trade-offs on either side (Google Inc. 2025). It also provides a simple method to request regular location updates making it suited to this use case.

Google's Maps API and Routes API was also considered to be coupled with the FusedLocationProviderClient, which would allow some of Google maps features to be utilized. Some useful features include the ability to determine alternative routes before the user takes the route, increasing the interval between updates as the route data could be extrapolated using Google maps' knowledge of the road and traffic data and other useful features. However, this uses an internet connection and violates the self-contained nature of the project. Furthermore, the APIs incurs a cost after a set number of requests per month and needs billing information to be set up to charge for uses over the limit. The limit was increased as of March 1st, 2025, which is a while after development for this project had started.

3.6.2. Location Data Acquisition

The interval between location updates were set at 20 seconds which was found to be acceptable in testing without violating the integrity of the algorithm. The balanced power mode was used for this application as the user's location needed to be as accurate as possible without causing significant battery drain on the user's device. The low power mode could not provide an acceptable level of accuracy while the high accuracy mode was found to utilize a substantial amount of battery power in testing.

Google requires transparency and proper disclosure when a device's location data is being retrieved. Background location access is significantly limited to a few times each hour to preserve (Google Inc. 2023). A Google does not specify a specific number of requests. If an application requires more frequent access, it must be brought to the foreground so that the user is aware. This can be done through a notification that remains as long as the location is being accessed. This allows the application to operate even though it is not actively on the screen and even when the screen is off. This also prevents the application from being automatically closed by some battery saving algorithms that a user's device.

3.6.3. User Alerting

The user is alerted by a notification after 30 consecutive abnormal points, which is equivalent to 10 minutes given that points are retrieved every 20 seconds. The notification has two action buttons that allow to user to state whether they are safe or not safe. When the user clicks the safe button, the notification is dismissed and the no alerts are triggered for the remainder of the route. If the user clicks not safe, an SOS alert is sent to the user's emergency contacts. The notification also dismisses itself if a point is found after the notification was sent. Notifications were sent using Android's Notification Manager API.

3.6.4. SOS Message

An SOS message is sent to the user's emergency contacts if the notification is left unattended for 15 minutes, the equivalent of 45 data points. The SMS included a google maps link to the user's current position. This location was taken with the high accuracy mode as to reduce the error of the location. As this was as infrequent, the battery consumption was not greatly affected. The message also was sent every 15 minutes until the user indicates that they are safe or until they stop moving for an extended period. This was done using Android's SMS Manager API.

3.6.5. Alert Snoozing

The application also gave the user the option to snooze the alerts for a period of the user's choosing. During this time, no alert notifications are sent to the user until the period is finished. During this time, the user's location is still tracked and saved automatically even if it is anomalous. This was implemented as users may know they are taking a different route and would not like to be disturbed.

4. Results

4.1. Testing Methodology

As stated in the previous section, each iteration of the application was tested after development. The modules of the application were unit tested, then integrated and tested again. Based on the errors or bugs discovered, the application was redesigned, developed and tested again till the results were satisfactory. When all the required modules were developed and tested, the app was then released for Beta Testing.

Some of the initial testing was done using the Android Emulator in Android Studio. This emulator allows for mock routes and locations to be simulated. The main device testing before beta release was done on a Samsung Galaxy S24 Ultra with 12Gb of RAM and 512Gb of storage. Its processor is a Snapdragon 8 Gen 3 with an 8-core CPU and has a 5000mAh battery. This device will be referred to as the main device. The Lockito application was installed on this to allow for simulation of routes on the mobile device. This is a mock location application that allows a device location to be simulated in a different place and even allowed routes to be simulated.

The debugger features such as watches and breakpoints included in Android Studio were used during testing to identify bugs.

4.1.1. Unit Testing

The unit tests were done using simulations of the modules with mock data or data that was already acquired. Some modules could not be tested as a single unit as it may have been built on the already developed application.

4.1.2. Integration Testing

After unit testing, the modules were integrated and tested on the Emulator at times but mainly on the device. Due to time constraints and the nature of the project which required physical movement for real world testing, the application was tested using Lockito before being further tested in a real-world application.

4.1.3. Alpha Testing

Alpha Testing involved testing the various iterations of the application in a real-world application using the main testing device. This testing was done by the developer and also involved the checking of the code syntax and logic. This testing started before the beta testing but also overlapped the beta testing due to time constraints. The code use for this testing is contained in the Test branch of the GitHub repository.

4.1.4. Beta Testing

The application was released to beta testers who had varying movement patterns and devices. The testers were instructed to go about their lives as they usually do while snoozing the alerts for 7 days. This was to allow the application to learn their general routes. Afterwards, app was allowed to run as normal. When bugs were detected, the application was redeveloped and updates were released. The beta testers had a modification of the application where their SOS messages was hard coded to be sent to the main testing device. Their version of the application also sent SMS messages to the main testing device when notifications were sent. The devices included a Samsung Galaxy S22, Samsung Galaxy A71, Samsung Galaxy A52, Samsung Galaxy A53 5G and a Xiaomi POCO X4 Pro 5G. The location data of the testers was not released for their privacy. The code use for this testing is contained in the BetaTest branch of the GitHub repository.

4.2.Traceability Matrix

This traceability matrix links the application and algorithm requirements to each other and to their respective test cases. The alpha and beta testing would by nature test all the requirements. The requirements listed for each test case may have been first tested at that point. However, requirements may have been tested additionally at later test cases especially when integration testing was done.

Table 6: Requirements Traceability Matrix

Test Case ID	Algorithm Requirement ID	Application Requirement ID
TC-LA-1	FR-AL-1	FR-AP-1 FR-AP-2 NR-AP-1
TC-RS-2	FR-AL-2 FR-AL-3 FR-AL-4	FR-AP-2
TC-SAD-3	FR-AL-5 NR-AL-3 NR-AL-6	-
TC-TAD-4	FR-AL-6 NR-AL-4 NR-AL-7	-
TC-NO-5	FR-AL-7	FR-AP-3 FR-AP-4
TC-SOS-6		FR-AP-5 FR-AP-6

TC-AS-7		NR-AP-7
Alpha Testing	NR-AL-1	
Beta Testing	NR-AL-2	
	NR-AL-5	
	NR-AL-6	
	NR-AP-2	
	NR-AP-3	
	NR-AP-4	
	NR-AP-5	
	NR-AP-6	

4.3.Location Acquisition Module

Test Case ID	TC-LA-1
Description of Module	This module consisted of the acquisition and storage of location points and location tracking with a regular and intervals. A button was implemented that printed the stored points back to the user.
Unit Testing Method	The module was tested by simulation on the Android Emulator and then on the main testing device in a real-world setting. Steps: 1. Click Button to start tracking 2. Move around for 3 minutes 3. Click button to print locations 4. Inspect coordinate points printed

	5. Repeat steps 2 to 4
Expected Results	• The locations with its timestamps should be printed when the button to print the locations is pressed. • The locations printed should be accurate to 50m. • There should be a 5 second interval between time stamps.
Unit Testing Results	The same location was stored and repeated when tracking on the emulator. The module worked as expected when tested on the main device.
Integration Testing Method	No Integration Testing as this was the first module.
Comments	The emulator's ability to simulate locations was unknown when this test was done, therefore both tests performed as expected.

4.4.Route Storage Module

Test Case ID	TC-RS-2
Description of Module	This module consisted of the tracking, segmentation and storage of routes. It also involved the reading of the stored routes into variables.
Unit Testing Method	No Unit Testing done as the module was built on existing Module.
Integration Testing Method	This module was tested using a prototype that allowed the user to record a route by pressing a button to start tracking the route

	and pressing a button to stop tracking at the end of the route. This was tested on the Android Emulator. Steps: 1. Set route to simulate in the emulator's location controls. 2. Click button to start tracking 3. Begin route simulation at 3X speed. 4. Utilise Debugger to ensure point are being tracked and stored 5. Allow simulation to run to completion and click button to stop tracking. 6. Click button to print route and Inspect route. 7. Repeat steps 1-6 for another route
Expected Results	• The route locations with its timestamps should be stored in the route point variables at regular intervals when inspected using the debugger watches and breakpoints • The tracking should stop when the button to stop tracking is pressed • All route locations with its timestamps should be printed when the button to print the locations is pressed.
Integration Testing Results	The module initially returned the same location point and did not update as the simulation moved.

Modifications	After research, it was found that this was a bug with the emulator. When the Google Maps application was opened on the emulator, the location was updated regularly.
Comments	Given the nature of Google Maps, it forces the location of the device to be updated every second and this allowed the developed application to access the up-to-date location.
Results after modification	The last route was stored as the first route and the last route. Both routes had the same points.
Modification	The Current Route data structure was reinitialised after the route was added to the saved routes data structure instead of all the points being deleted from current route.
Results after modification	The module performed as expected
Comments	The current route structure when added to the saved routes, stayed linked to the added route. Therefore, the changes made to the current route was reflected in all the routes added to the saved routes as they all pointed to the same structure and area of memory.

4.5.Spatial Anomaly Detection Module

Test Case ID	TC-SAD-3
Description of Module	This module was responsible for the detection of spatial anomalies using the algorithm developed.
Unit Testing Method	This module was built on the existing module therefore unit testing was not explicitly done.

Integration Testing Method	This module was tested using an improvement of the prototype used in the previous module. This module allowed the user to select one of the already stored routes (See Appendix A). This route selected was considered the normal data and user's location was tested to determine anomalies. This was tested on the Android Emulator. Steps: 1. Select Route 1 to simulate in the emulator's location controls. 2. Click button to begin tracking and select Route 1. 3. Begin route simulation at 3X speed and open Google Maps application. 4. Utilise Debugger breakpoint to identify points that the algorithm considers anomalous. 5. Record and inspect any anomalous data 6. Repeat steps 1-5 using Route 2 7. Repeat steps 1-5 using Route 3 8. Repeat steps 1-5 simulating Route 2 while still selecting route 1
Expected Results	• When the simulated route is the same as the selected route, the module should return no anomalous points • When the simulated route and the selected route varies, the module should return anomalous points only when the simulated route varies from the selected route.

Integration Testing Results	The module initially returned some false negatives for the same simulated and selected routes and false positives for the different simulated and selected routes.
Modifications	The y-intercept threshold was refined to ± 0.001 and the bounds test was added to the algorithm. The bound test threshold was also refined to ± 0.0015 .
Results after modification	The module performed as expected.
Comments	The algorithm initially did not include the bounds test which was added only after testing.

4.6.Temporal Anomaly Detection Module

Test Case ID	TC-TAD-4
Description of Module	This module was responsible for the detection of temporal anomalies using the developed algorithm after the spatial check was done.
Unit Testing Method	This module was built on the spatial module therefore unit testing was not explicitly done.
Integration Testing Method	This module was tested using an iteration of the application. This iteration allowed for routes to be learnt and saved. The incoming data was tested against all the saved routes. This was tested on the main device using the Lockito application and changing the time of the device. Steps: 1. Select Route 4 to simulate in the Lockito application.

	2. Click button to start tracking. 3. Begin route simulation at 2X speed and note date and time of the start of the simulation. This time is referred to T0. 4. Allow simulation to run to completion and allow time for application to save the route. 5. Change device's time back to T0 and restart simulation. 6. Utilise Debugger breakpoint to identify points that the algorithm considers anomalous. 7. Record and inspect any anomalous data. 8. Repeat steps 5-7 using the device time as T0 + 5 minutes 9. Repeat steps 5-7 using the device time as T0 + 30 minutes
Expected Results	• The module should return no anomalous points when the device time is T0 and T0 + 5 minutes. • The module should return anomalous points when the device time is T0 + 30 minutes
Integration Testing Results	The module performed as expected.
Comments	This test did not account for all scenarios and was not as comprehensive as required. The rigorous testing of this module was done later in the beta testing.

4.7.Notification Module

Test Case ID	TC-NO-5
Description of Module	This module was responsible for sending notifications to user. The notification also allowed for user interaction for them to indicate whether they are safe or not.
Unit Testing Method	This module was tested using a prototype that sent a notification when a button in the application was pressed. The action buttons were also tested to see if the application recognised when it was clicked. Steps: 1. Click button to send notification. 2. Verify notification was sent and included intended information and buttons. 3. Click “Safe” action button 4. Verify variable was changed using debugger 5. Click “Not Safe” action button 6. Verify variable was changed using debugger
Expected Results	• The notification should appear when button is pressed with respective sound alert. • The notification should have two action buttons: “Safe” and “Not Safe” • A variable in the app should change to “Safe” or “Not Safe” when the respective action button is pressed.
Unit Testing Results	The module performed as expected.

Integration Testing Method	This testing was done using the developed application on the main device. It involved simulating a new route using the Lockito application to allow the notification to be sent and ensuring that the action buttons function as expected. Steps: 1. Select a new route to be simulated. 2. Start tracking in application and begin the simulation 3. Wait for notification alert 4. Click “Not safe” action button and use debugger to verify variable change 5. Click “Safe” action button and use debugger to verify variable change 6. Stop the simulation and allow route to be saved. 7. Restart simulation using the same route at the same time. 8. Ensure that no notification is sent
Expected Results	• The notification should appear after 30 anomalous points or 10 minutes as • A variable in the app should change to “Safe” or “Not Safe” when the respective action button is pressed. • No Notification should be triggered when the route is simulated a second time
Integrated Testing Results	The module performed as expected.

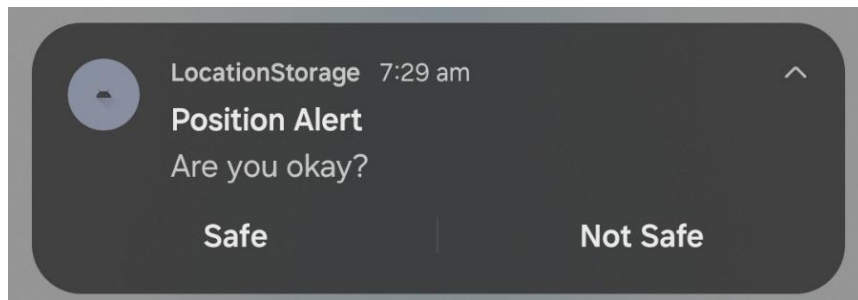


Figure 5: Screenshot of Notification

4.8.SOS Message Module

Test Case ID	TC-SOS-6
Description of Module	This module was responsible for sending SOS messages to user's emergency contact. The message possessed a google maps link that pointed to the user's location at the point that the message was sent. This module was also responsible for the addition of emergency contacts.
Unit Testing Method	This module was tested using a prototype that sent a message when a button in the application was pressed. Steps: 1. Add an emergency contact using the application dialog 2. Verify contact was added using "Print emergency Contacts" button. 3. Repeat steps 1 and 2 for another emergency contact 4. Click button to send SOS message. 5. Verify message was sent with the intended content to intended contact. 6. Click on link with user's location

	7. Verify that location was accurate
Expected Results	- Emergency Contacts' name and number should be printed to the user when button is clicked. - The message should be sent to the intended emergency contacts. - The message should contain the name of the contact and a google maps link to the user's location - When the link is clicked, a Google map with a red marker indicating the user's position should be opened.
Unit Testing Results	The user's location coordinates were not sent in the message, instead, (0.0,0.0) was sent (Figure 6). This pointed to a location in the South Atlantic Ocean.
Modifications	The message was split into two parts then sent as one message using the sendMultipartTextMessage function of the SMS Manager API.
Results after modifications	The module performed as expected.
Comments	Message sent using the sendMessage function had a character limit of 160 characters when using GSM-7 characters but only 70 characters when using Unicode characters as Unicode characters use more bits. Since the SOS message contained Unicode characters, it went over the limit and produced unexpected results.

Integration Testing Method	This testing was done using the developed application on the main device. It involved simulating a new route using the Lockito application and ignoring the notification sent to allow the message to be sent to the emergency contacts. The application also had the ability to add emergency contacts at this iteration. Steps: 1. Select a new route to be simulated. 2. Start tracking in application and begin the simulation 3. Wait for notification alert and SOS message to be sent 4. Click on the link in the message and verify its behaviour. 5. Click “Not Safe” action button of the notification and verify that another SOS message was sent 6. Click “Safe” and verify no more SOS message were sent for the remainder of the route 7. Stop the simulation
Expected Results	• The notification should appear after 30 anomalous points and the SOS message should be sent 45 points after this. • The message should have a link that points to the user’s location. • Clicking the link should take you to the user’s location in Google Maps and place a red marker on the area.

	• Another SOS message should be sent when the user clicks the “Not Safe” action button on the notification • No further SOS messages should be sent when the user clicks the “Safe” action button.
Integrated Testing Results	SOS messages continued to be sent after the user clicked safe.
Comments	The variable that indicated if a notification was active was reset before the application could service the function that set the variable to stop the notifications.
Modification	The variable was reset after the function was serviced.
Results after modification	Application performed as expected.

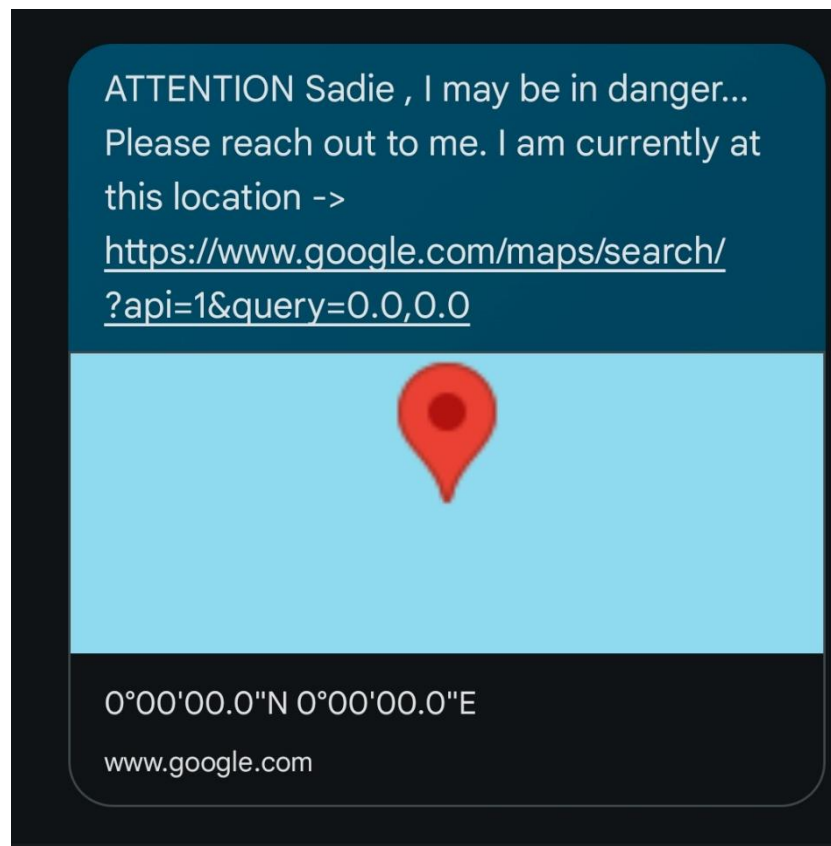


Figure 6: Screenshot of SOS message with (0.0,0.0) Location

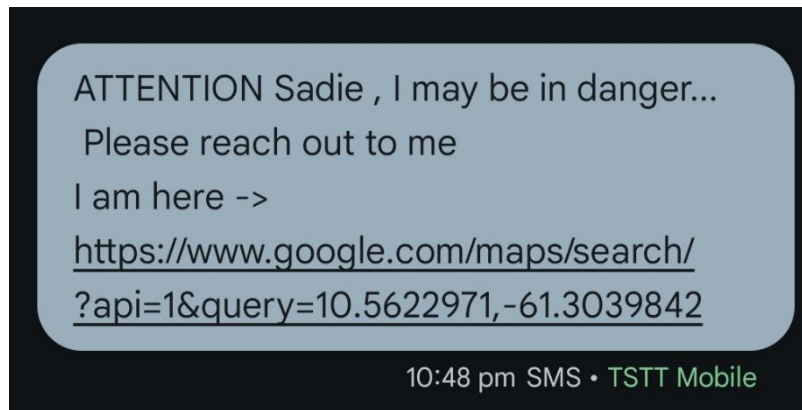


Figure 7: Screenshot of SOS message working as expected.

4.9.Alert Snoozing Module

Test Case ID	TC-AS-7
Description of Module	This module was responsible for the snoozing of the alerts to the user and SOS messages for a set period of time.
Unit Testing Method	This module builds on the functionality of the other modules. therefore, unit testing was not done.
Integration Testing Method	This module was tested using an iteration of the application. This iteration contained all the previously mentioned modules. It was tested using the Lockito application and main device. Steps: 1. Select a new route to simulate in the Lockito application. 2. Snooze alerts for 15 minutes using application Dialog. 3. Click button to start tracking. 4. Begin route simulation at 2X speed. 5. Allow simulation to run to completion and ensure no alerts are sent.

	6. Allow time for the route to be saved and ensure route is saved. 7. Repeat steps 1 to 6 for another route
Expected Results	- No Notifications should be sent to the user - The route should be saved after the necessary time has passed.
Integration Testing Results	Snooze timer stayed at 15 minutes for entirety of the test.
Comments	Code to decrement snooze timer was omitted in error
Modification	Code to decrement timer was added
Results after Modification	Module performed as expected
Comments	This test was basic and more rigorous testing of this module was done later in the beta testing.

Specify Snooze Time

30 24 60

Days: 0 Hours: 0 Minutes: 0

1 1 1

CANCEL CONFIRM

Figure 8: Screenshot of Application Dialog to Snooze Alerts

4.10. Alpha Testing

Error ID	Bug/Error	Reasoning	Modification	Bug/Error Solved?
TC-AT-1	Temporal Clustering did not function correctly	Incorrect bitwise “OR” Operator was used in code	Correct Operator “ ” was used	Yes
TC-AT-2	Temporal clustering did not occur when user moved right after starting the tracking or if the user was already moving while starting the tracking.	Temporal Clustering was done only when the incoming location point was near to the last and there was only one point in the route. i.e. the user was not moving	Temporal clustering was added at the first point in the route as well	Yes
TC-AT-3	Friday was considered the weekend by the application	Human Error in coding	The code was corrected to consider Friday as a weekday	Yes
TC-AT-4	Route was ended and saved at a traffic light	Interval was shortened for testing purposes and was not reinstated back to 20 seconds	Interval changed back to 20 seconds	Yes

TC-AT-5	Location in SOS message was inaccurate by more than 100 metres	At this point, the SOS message did not get a current location at high accuracy but used the last location update which was in Balanced power mode	The current location taken at high accuracy was sent in the link	Yes
TC-AT-6	While traversing a saved route about 10 minutes later than usual, with denser traffic than usual, a notification was sent, indicating that the route was not known by the application.	The route error was omitted in some areas of temporal testing in error.	The route error was included in the calculations for all situations.	Yes
TC-AT-7	Temporal error calculation did not account for when the TEV + PT may exceed the logical bounds (>1439 or <0)	This was overlooked when designing and coding.	The addition of 1440 and the modulo of 1439 were used to account for these boundaries.	Yes

4.11. Beta Testing

Error ID	Bug/Error	Reasoning	Modification	Bug/Error Solved?
TC-BT-1	No location updates were recorded and the application was killed after a few hours	User had a battery saver enabled on their phone. The battery saver limited all background usage	Battery Saver was switched off	Partially (The application was not killed but updates were less than expected)
TC-BT-2	Notification was triggered when the user arrived at their destination.	The variable that indicated whether a point was found was reset to false at the end of every check. This meant that if the user stopped moving, the application would recognise the user as off route as the	The variable was reset to false right before a new check was done	Yes

		variable was false. This would trigger the notification and SOS message.		
TC-BT-3	There was no way to indicate “Safe” if phone was in DND or if the notification was swiped away in error	This was overlooked in design and development	“I am Safe” button in application was implement in the application	Yes
TC-BT-4	Notifications were not sent to the user after a few hours.	The application was being restricted by the OS in effort to save battery.	1. The Foreground service type was declared at runtime 2. The application was set to “Unrestricted” in the Battery Optimization Settings 3. The application was put on the “Never Auto Sleeping” List	1. No 2. No 3. No 4. No 5. No 6. No

			4. Background location permissions were added and allowed 5. Batched location update feature was implemented 6. Foreground Service was implemented with the START_STICKY flag	
TC-BT-5	Snooze timer went for approximately double the time it should.	This was also due to the application not being restricted in the background. Therefore, it was not able to decrement the snooze timer as necessary		

4.12. Battery Consumption

The application on balanced mode used less than 1% in 24 hours on the main testing device.

On the beta Testing devices, it also used less than 1%.



Figure 9: Screenshot of Application Percentage Battery Usage on Balanced Power mode

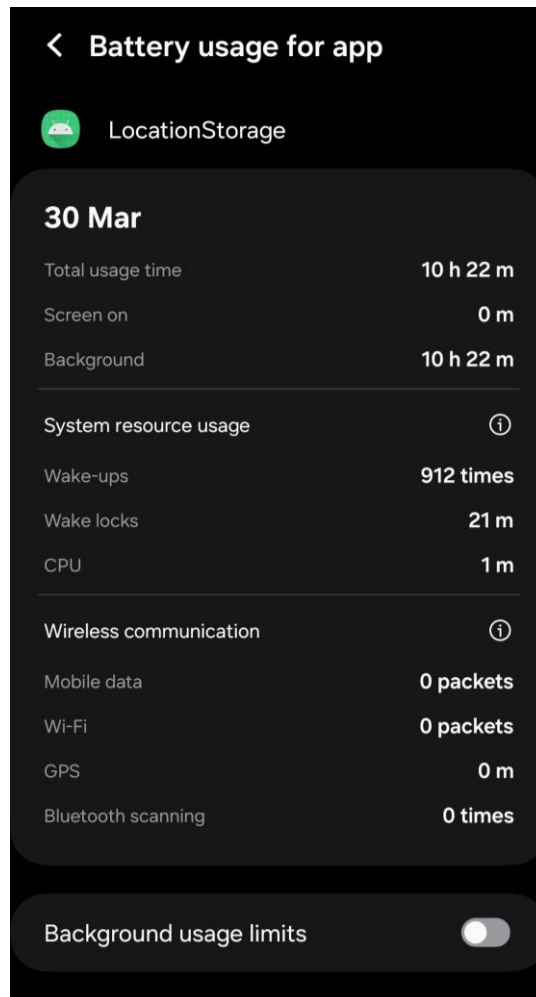


Figure 10: Screenshot showing Breakdown of Battery Usage

The application was also tested for three days on high accuracy mode which consumed around 18% one day and 21% another day.

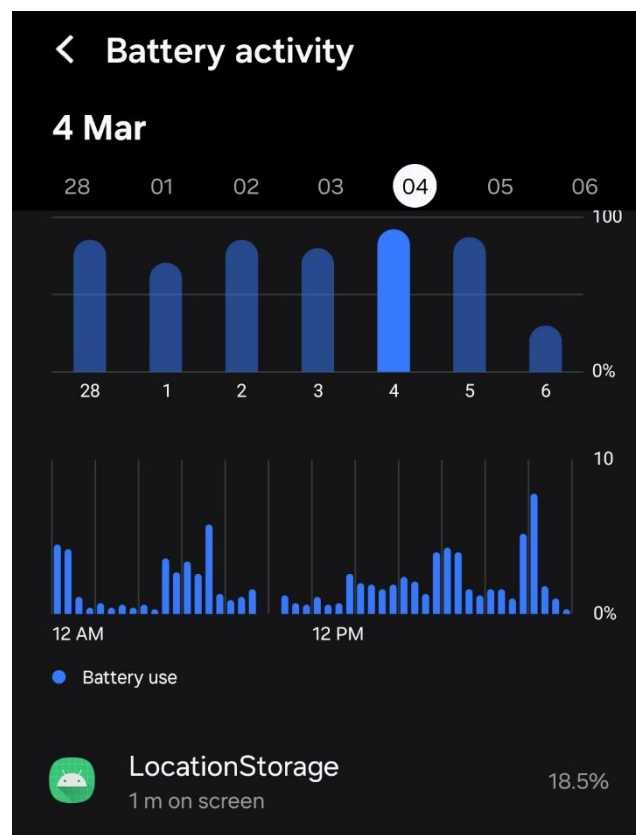
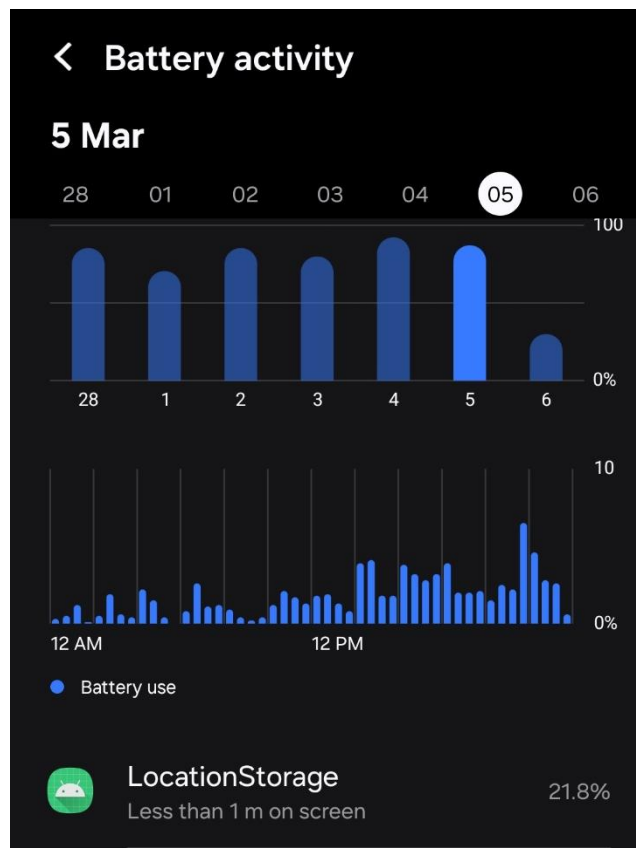


Figure 11: Screenshots showing Application Percentage Battery Usage in High Accuracy Mode

4.13. Storage Usage

The application used approximately 10 MB of Storage after 3 weeks on the main device. The main device had over 80 saved routes at this point.

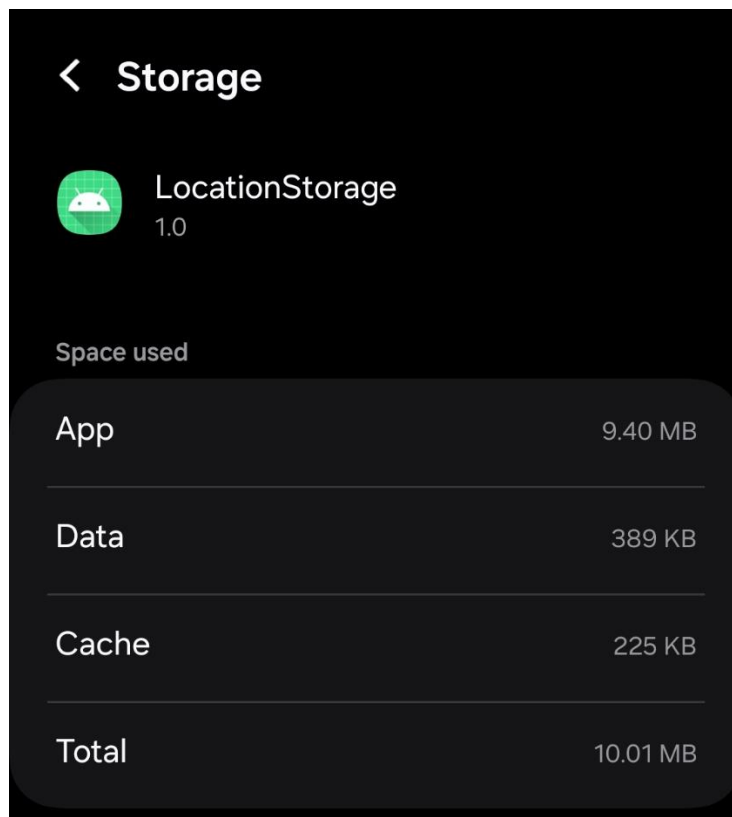


Figure 12: Screenshot of Application Storage Usage

The application itself was 9.40 MB and the data is stored, which was the location data, was only 389KB after this time.

5. Discussion

The algorithm developed performed as expected and achieved the requirements of the project. However, the application's performance was stymied by the Android OS in its effort to optimize and save battery life. As also seen in testing, having the battery saver feature activated stopped all functionality of the application. Seeing that the application performed as expected on the main testing device and for a day on the beta testing devices, it showed that the error lies with how the application was developed and not the logic or performance of the spatiotemporal anomaly detection algorithm itself.

5.1.Spatiotemporal Anomaly Detection Algorithm Sufficiency

The algorithm designed performed as expected when the application was allowed to run before being subjected to restrictions of the battery saving features of the Android OS. The rigorous spatial testing of the algorithm ensured that the logic and thresholds used in the developing the algorithm was refined to the point where the algorithm rarely returned false readings.

The temporal aspect was tested less rigorously during the unit and integration testing due to deadlines set for the application to be released for alpha and beta testing. However, the module tested rigorously during the alpha testing. During this phase of testing, it was subjected to the varying departure times of the tester to his place of work and back. The error with notification triggering incorrectly when the tester departed within 10 minutes of the latest recorded departure time was found to be due to an error in coding as mentioned in the Testing section. With the TEV implemented, this should not have been triggered as it should account for the variations in traffic and departure times. When this was corrected, the module then performed as expected. When the departure time varied by more than 15 minutes, a notification was correctly issued. When the departure time was earlier and later than the normal times but within 15 minutes, the notification was not issued as expected. The same route was traversed every

day. When a detour was taken for longer than 10 minutes, a notification was issued as expected. This behaviour was also replicated on the beta testing devices when the application performed as expected.

The implementation of the algorithm could tolerate a few intermittent false negatives readings as it would require consistent false readings for an extended period for the application to trigger notification in error. A false positive however, could extend the notification time significantly and in real life scenarios may further endanger the safety of the users. Nevertheless, false positives may be very rare as it will be required to pass both spatial and temporal checks.

5.1.1. Algorithm Improvements

A possible improvement of the algorithm could be to tolerate some intermittent false positives. This can be done by requiring a series of around 5 to 10 positive readings before the algorithm recognised the route as being normal.

Another improvement can be the spatial clustering of the routes. New routes that are found to be normal should not be saved as a new route but the existing route should be improved with any new data. If the percentage of route points found on an existing route was above a certain threshold, the existing route should be improved by inserting the points that were not found. The times of the new route should also be used to improve the route either by extended the thresholds used for that route or by creating an alternate timeline. The alternate timeline should only be created when the time varies above a certain threshold which should be less than the 15 minutes. This will reduce the storage utilized by the application and reduce the computational expense as the application will have to compare against less routes.

Another improvement can be taking into consideration the frequency at which routes are taken. This can help the algorithm understand the common routes at common times and allow the algorithm to issue notifications more intelligently. However, this would require much data to

function satisfactorily and would be time consuming to tune and refine without the use of Artificial Intelligence.

The algorithm should also test when the user is not moving. An indication that a user may be unsafe can be when they stay at the same location for an unnaturally extended period. The algorithm currently only tests when the user has moved or is moving. The current implementation of the algorithm will not detect an anomaly if a user gets in an accident while traversing a normal route. The algorithm should also test when the user remains at a known location for a period significantly longer than usual.

5.2.Battery Optimization Issues

The behaviour of the application on the beta testing devices was unexpected as the application functioned as expected for over a week on the main testing device throughout the alpha testing. However, the issue also occurred on the main device after it was not opened for a few days. This discrepancy may be due to the reasons itemized in the following subsections.

5.2.1. Android's Adaptive Battery

Android and Samsung utilize an adaptive battery algorithm which is designed to learn a user's application usage pattern and put applications to sleep or keep them running at certain times. Since the application was interacted with considerably more on the main testing device, the application was kept running and was not put to sleep. On the beta testing devices however, the application was most likely installed, started and never opened afterwards. This meant that the battery saving algorithm recognised the application as low priority and put to sleep after some time. Interestingly, the Xiaomi POCO did not put the application to sleep for over 10 days while on Samsung, it was put to sleep after a day. Therefore, Samsung's battery saving algorithm may be more restrictive than others, causing the application to malfunction.

To counteract this, the application was set to “Unrestricted” in the application’s battery settings. This is Google’s implementation that allows an application to run in the background without any restriction. However, this allowed the application to run unrestricted for a few hours to a day but then the application begun malfunction with locations updates only being recorded at the start and end of routes or at erratic intervals significantly greater than 20 seconds.

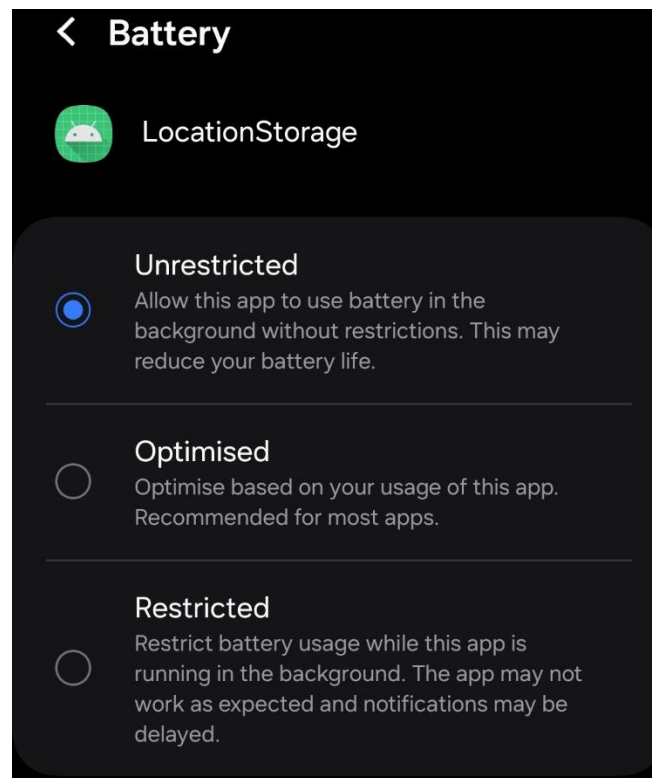


Figure 13: Screenshot of Unrestricted Battery Setting

The application was then added to the “Never Auto Sleeping” List on some of the testing devices. This is Samsung’s implementation of allowing an application to run in the background without being put to sleep. The change was only made on some devices to provide a control for the test case. The behaviour of the application was the same as when it was set as unrestricted. No application can be both “Unrestricted” and on the “Never Auto Sleeping” List as soon as either is selected, the other is removed.



Figure 14: Screenshot showing Application added to "Never Auto Sleeping" List

5.2.2. Capacity of phones

Another reason that was looked at was the difference in the capacity of the phones. The main device had the best specifications with the largest RAM and battery capacity and the quickest processing speed. The phone that performed the worst was the A52 which had the most limiting specifications. Therefore, the app will have been killed faster or subject to the most restrictions on this phone due to the lower limits. Since this issue presented itself on all the devices, this was not the root cause. However, this shows that the issue may be linked to some battery or memory saving process.

5.3.Frequency of Location Updates

The application used a foreground service which was stated to be killed very rarely based on Google's documentation (Google Inc. 2025). After the bypassing of the battery optimization proved futile, the focus was shifted to see if the problem lied with the location updates. As stated in a previous subsection, the location updates recorded pointed to the start and end of the routes. This would have meant that the application was not killed by the OS but may not have been able to run as required or the location was not updated as required. Therefore, issue that may have prevent regular updates were looked at.

5.3.1. Background Location Access

The application initially did not request background location as it was limited to a few location updates an hour which was far less than required for this application. However, after some research it was seen that other applications had this issue and Google stated that if location access was required for an indefinite period of time, background location access was required. This allowed the user to give the user the option to allow location access all the time. However, the problem persisted with this implementation. This may be due to the background location access limits.

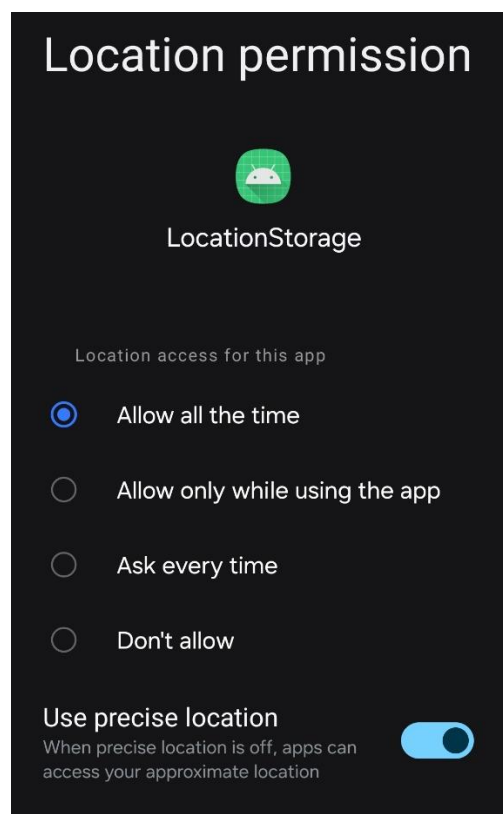


Figure 15: Screenshot showing Background Location Permission

5.3.2. Location batching

The Fused Location Provider API allows for batched location updates which allows for more time-frequent location updates in the background (Google Inc. 2024). The location updates are batched which means that while the application may wake up only 10 times in an hour, it may

receive 50 location updates as the application receives batches of 5 updates every time it wakes. This allows more updates while staying within the limits of the background location access. However, when this was implemented into the application, the issue was still experienced.

5.3.3. Data connection

As stated before, the Fused Location Provider API uses a blend of Wi-Fi, network and GPS to achieve its location updates while minimizing power consumption. In the balanced power mode, the API utilizes more of Wi-Fi and network access as GPS significantly drains the battery. As a user is travelling, he is unlikely to have Wi-Fi access, which means that network access is predominantly used. When a user has data enabled, this may allow more location updates as the phone may be required to communicate with cell towers more frequently. However, no documentation was found to substantiate this theory.

Additionally, having data enabled may allow other applications such as Google Maps request location updates in the background, making them available for the application to use. A beta tester also stated that they believe the application mainly functioned as expected when they had data enabled or when they had Wi-Fi access. They stated that when they got home then they would see the notification. This is a probable theory as Wi-Fi access grants an inexpensive location update and this may be why the starts and ends of a route may have been the only ones recorded as the users had Wi-Fi access in both areas. However, the beta tester may have only recognised the notification when he arrived at his location as this may be when they would have paid attention to their phone. Interestingly, the main device and the Xiaomi device both had data access constantly enabled. These two devices also performed the best and did have the issue with only the start and end of routes being recorded. This fact may confirm that data access may improve the performance of the application.

5.3.4. Location Update Mode/Priority

Following from the previous subsection, the location update mode may have limited the performance of the application. The high accuracy mode of the API utilizes GPS more often and guarantees more location updates. This may solve the issue with the location updates occurring only at the start and end of the routes. However, the high accuracy mode utilizes significant battery power as was tested in the alpha testing. Therefore, the high accuracy mode was not tested in beta testing as this may have inconvenienced the testers. There were also time constraints as this would take more than two days to test as the application performs as expected for about a day before being restricted or killed. Additionally, as testers are not guaranteed to take a new route every day and may not have the time to willingly take a new route, this testing may have taken longer.

5.4.START_STICKY Flag

After all the previous updates and implementations, the START_STICKY flag was used with the foreground serviced only for the A52 device. This flag allows the service to restart itself if it was killed by the OS or if the phone was restarted. However, the service is not guaranteed to restart instantly. It may be restarted a while after when the OS decides that the unused memory of the phone is sufficient.

This kept the service running for three consecutive days on this device with location updates occurring more frequently but not as frequent as required. Notifications also were not triggered as they should be. However, this flag may prove important to solving the issue experienced. The flag was only implemented on the A52 device as it proved to be the most difficult device. Implementing on one device also provided a control in the test case.

5.5.Possible Solutions

One possible solution may be the implementation of Geofences in conjunction with location updates using the high accuracy mode. A geofence is a virtual geospatial boundary that allows alerts to be triggered when a user leaves, enters or stays within the boundary. Android possess a Geofencing API that significantly optimizes battery usage and is allowed to access location updates more frequently than the Fused Location Provider (FLP) Client (Google Inc. 2024).

The possible solution would involve the creating of geofences when the user is not moving for an extended period like the period used to signal the end of the route. When the geofence is created, the location updating is suspended. When the alert triggers that the user has left the geofence, the location updating should be triggered with the high accuracy mode enabled. This would guarantee more location updates with high accuracy mode while reducing the battery drain as the updates will not be constantly running throughout the day. The more efficient geofencing API will be running for the majority of the time. This solution however could not be developed and implemented within the time constraints of the project.

Another solution may be instructing the users to at least open the application once a day to ensure that the service is allowed to run more freely.

5.6.Alert Snoozing Module

The alert snoozing module also did not function as expected on any device. While the alerts were snoozed successfully, the snooze time ran significantly longer on the devices. When the application performed as expected on the main device, the alert snooze time still lasted up to twice as long as the set time. This is due to the timer using the location updates to decrement the timer. Since it was proved that the updates do not occur as often as required, this module also malfunctioned.

A possible solution to this problem will be using the date and time of the device to determine when to stop the snoozing of the alerts. This will bypass the unreliability of the location updates. This solution also was not implemented due to time constraints and the focus being on getting the location updates more consistently.

5.7.Application Improvements

The application could allow the user to determine how long they want the application to wait before sending notifications and alerts. As various users will have differing use cases, they may want to utilise different timings thresholds. They can also be allowed to modify the contents of the SOS message as they see fit.

Parental controls can also be implemented in the application to restrict children in the care of their parents from turning off the tracking or snoozing alerts by mistake or without their parents' permission. The clicking of the "Safe" and "Not Safe" action button could also be made to be two factor to prevent accidental touches. This improvement was suggested by a beta tester. This can also be taken a step further, by requiring biometric or password authentication, when clicking the buttons to ensure the user alone can indicate their safety. The user should be allowed to turn this feature on and off if required.

6. Conclusion

The algorithm satisfied its requirements as it identified the anomalous data with sufficient accuracy. Therefore, the application produced no notifications in error. Notifications were also triggered as necessary when the application was allowed to run as required by the OS. However, the application was not allowed to function as required if it was not opened at least once a day. This was due to the stringent battery saving processes of the Android OS and its limits on access to a user's location data. The efforts to bypass these were not successful and the implementation of the application will have to be redesigned to achieve the desirable performance.

References

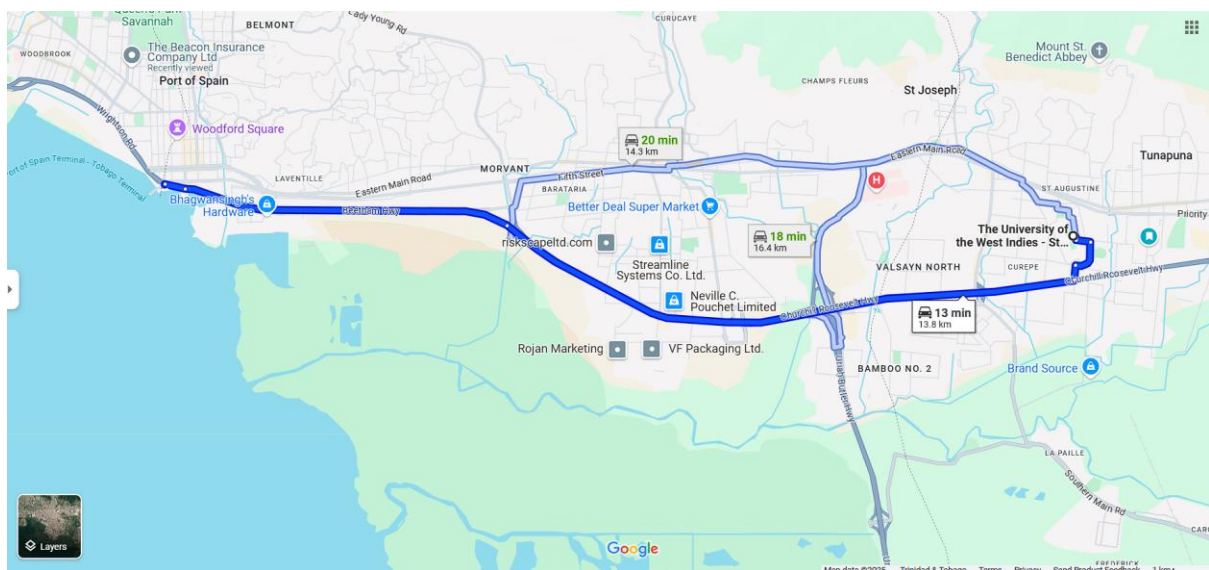
- Asres, Mulugeta Weldezigina, Christian Walter Omlin, Long Wang, David Yu, Pavel Parygin, Jay Dittmann, Georgia Karapostoli, et al. 2023. "Spatio-Temporal Anomaly Detection with Graph Networks for Data Quality Monitoring of the Hadron Calorimeter." *Sensors* 1-23.
- Bergmann, Dave. 2023. *What is semi-supervised learning?* 12 December. Accessed March 9, 2025. <https://www.ibm.com/think/topics/semi-supervised-learning>.
- Collier, Kevin. 2025. *A company that tracks and sells Americans' location data has seemingly been hacked.* 10 January. Accessed April 1, 2025. <https://www.nbcnews.com/tech/security/location-data-broker-gravy-analytics-was-seemingly-hacked-experts-say-rcna187038>.
- Daily Express. 2024. "A Growing Problem of Disappearances." *Daily Express*, 8 December. https://trinidadexpress.com/opinion/editorials/a-growing-problem-of-disappearances/article_b5d9ab2e-b5bd-11ef-89c0-cb96c0e9a5c4.html.
- Eldawy, Eman O., and Hoda M.O. Mokhtar. 2020. "Clustering-Based Trajectory Outlier Detection." *International Journal of Advanced Computer Science and Applications* 133-139.
- Federal Trade Commission. 2025. *FTC Finalizes Order Prohibiting Gravy Analytics, Venntel from Selling Sensitive Location Data.* 14 January. Accessed April 1, 2025. <https://www.ftc.gov/news-events/news/press-releases/2025/01/ftc-finalizes-order-prohibiting-gravy-analytics-venntel-selling-sensitive-location-data>.
- Google Inc. 2023. *Access location in the background.* 1 November. Accessed March 16, 2025. <https://developer.android.com/develop/sensors-and-location/location/background>.
- . 2024. *Android's Kotlin First Approach.* 27 June. Accessed April 1, 2025. <https://developer.android.com/kotlin/first>.
- . 2024. *Background Location Limits.* 29 April. Accessed March 25, 2025. <https://developer.android.com/about/versions/oreo/background-location-limits>.
- . 2025. *Get the last known location.* 4 March . Accessed March 16, 2025. <https://developer.android.com/develop/sensors-and-location/location/retrieve-current>.
- . 2025. *Request Location Permissions.* 4 March. Accessed March 9, 2025. <https://developer.android.com/develop/sensors-and-location/location/permissions>.
- . 2025. *Services Overview.* 28 January. Accessed March 27, 2024. <https://developer.android.com/develop/background-work/services>.
- International Committee of the Red Cross. 2024. *Africa: Cases of missing persons registered with ICRC rise 75 per cent in five years.* 29 August. Accessed February 27, 2025. <https://www.icrc.org/en/news-release/africa-cases-missing-persons-registered-rise-75-percent-in-five-years>.

- JetBrains. 2025. *FAQ*. 5 March. Accessed April 1, 2025. <https://kotlinlang.org/docs/faq.html>.
- Karadayi, Yildiz, Mehmet N Aydin, and Arif Selcuk Ogrenci. 2020. "Unsupervised Anomaly Detection in Multivariate Spatio-Temporal Data Using Deep Learning: Early Detection of COVID-19 Outbreak in Italy." *IEEE Access*.
- Nawaratne, Rashmika, Damminda Alahakoon, Daswin De Silva, and Xinghuo Yu. 2019. "Spatiotemporal Anomaly Detection Using Deep Learning for Real-Time Video Surveillance." *IEEE Transactions on Industrial Informatics* 393-402.
- Pokrajac, Dragoljub, Aleksandar Lazarevic, and Longin Jan Latecki. 2007. "Incremental Local Outlier Detection for Data Streams." *IEEE Symposium on Computational Intelligence and Data Mining* 504-515.
- Salehi, Mahsa, Christopher Leckie, James C. Bezdek, Tharshan Vaithianathan, and Xuyun Zhang. 2016. "Fast Memory Efficient Local Outlier Detection in Data Streams." *IEEE Transactions on Knowledge and Data Engineering* 3246-3260.
- Sedaghat, Leela, John Hersey, and Michael P. McGuire. 2013. "Detecting Spatio-Temporal Outliers in Crowdsourced Bathymetry Data." *GEOCROWD '13: Proceedings of the Second ACM SIGSPATIAL International Workshop on Crowdsourced and Volunteered Geographic Information*. New York: Association for Computing Machinery. 55-62.
- Tian, Zhiwen, Ming Zhuo, Leyuan Liu, Junyi Chen, and Shijie Zhou. 2023. "Anomaly detection using spatial and temporal information in multivariate time series." *Scientific Reports*. <https://doi.org/10.1038/s41598-023-31193-8>.
- UNICEF. 2023. *Kidnappings of children and women spiking at alarming rates in Haiti*. 7 August. Accessed February 27, 2025. <https://www.unicef.org/press-releases/kidnappings-children-and-women-spiking-alarming-rates-haiti#:~:text=The%20latest%20reports%20received%20by%20UNICEF%20reveal,and%20used%20for%20financial%20or%20tactical%20gains>.
- Wei, Zhaokun, Xinlian Xie, and Xiaoju Zhang. 2022. "Maritime anomaly detection based on a support vector machine." *Optimization* 11553-11566.
- Wu, Chunrui, and Junfeng Tian. 2020. "Spatio-Temporal Outlier Detection: A Survey of Methods." *International Journal of Frontiers in Engineering Technology* 106-120.
- Yan, Zhu, Li Hongwei, Fan Chao, Xu Donghao, and Shi Fanglin. 2017. "Clustering Based Taxi Trajectory Outlier Detection." *Computer Engineering* 16-20.
- Yao, Lingling, and Zhanquan Wang. 2015. "Research on the Algorithm of Hadoop-Based Spatial-Temporal Outlier Detection." *2015 Fifth International Conference on Instrumentation and Measurement, Computer, Communication and Control (IMCCC)* 799-804.
- Zhang, Zhongping, Yuehan Hou, Yin Jia, and Ruibo Zhang. 2025. "An outlier detection algorithm based on local density feedback." *Knowledge and Information Systems*.

Appendix A: Routes for Spatial Testing

Route 1

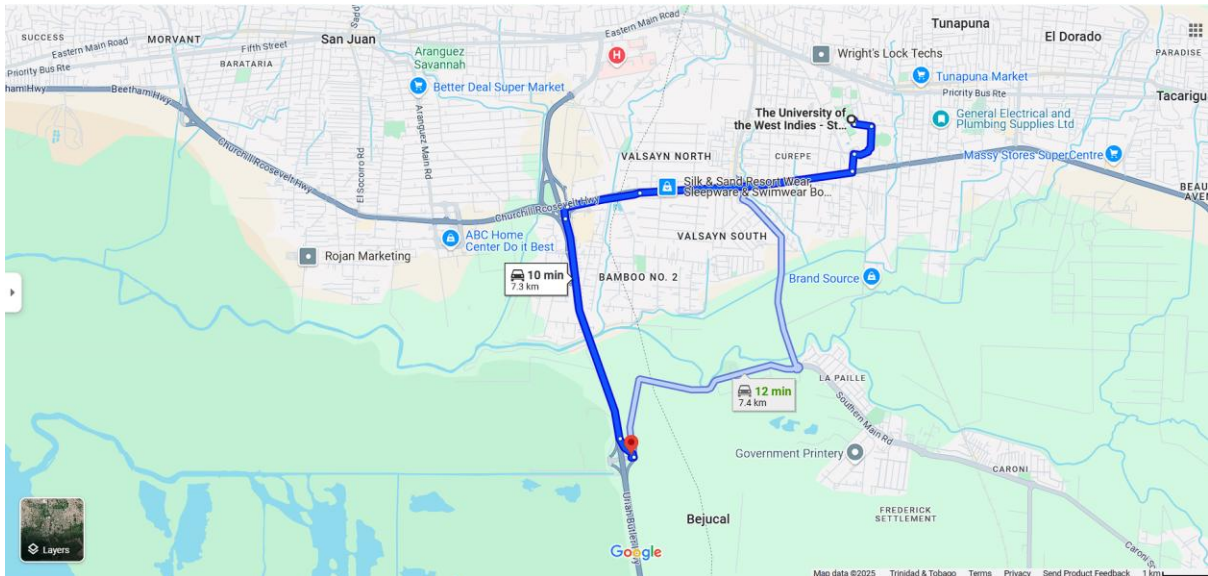
This route represented a simple route with predominantly straight lines and not many turns. The route starts at the University of the West Indies – St Augustine Campus and ends at the Inter-Island Ferry Terminal and is highlighted in dark blue.



Google Maps image of Route 1

Route 2

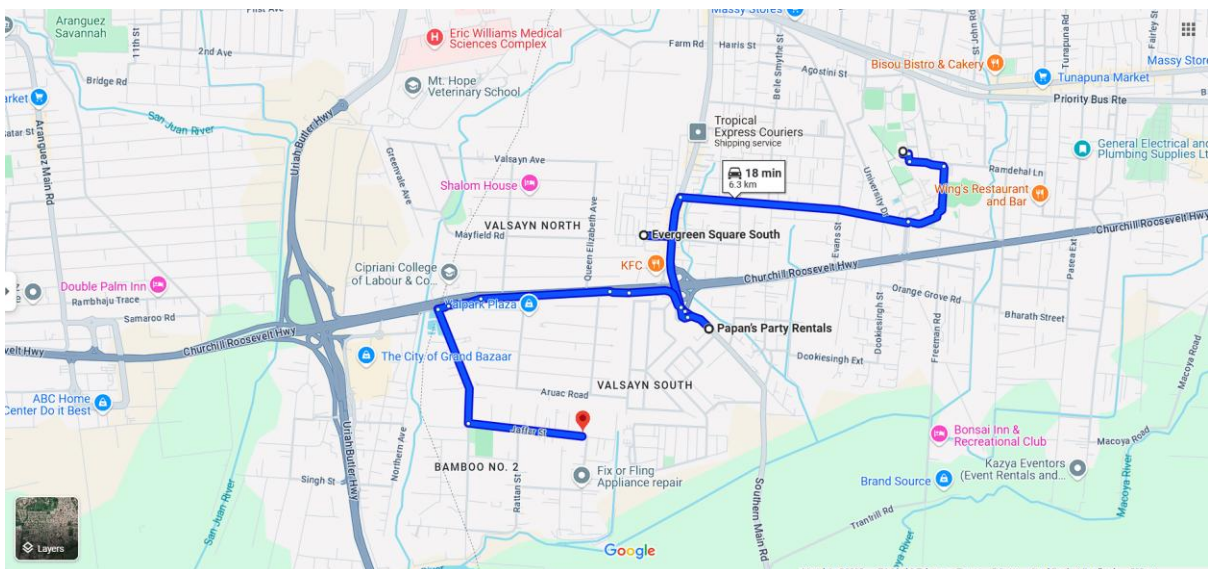
This route also represented a simple route but however was selected due to the verticality of the roads. This was to ensure that no bugs were encountered as the gradients of lines became significantly larger. The route also begins at the University of the West Indies – St Augustine Campus and ends at the Caroni Flyover.



: Google Maps Image of Route 2

Route 3

This route represented a convoluted route with many turns. It started at the University of the West Indies – St Augustine Campus, went to Evergreen Square S, visited Papan’s Party Rentals and finally ended at Ravi’s Auto Industries.



Google Maps Image for Route 3